RTS3917 Series Crypto Secure Boot User Guide

Release v5.3

Realtek

Oct 09, 2025

COPYRIGHT

DISCLAIMER

Realtek provides this document ``as is'', without warranty of any kind, neither expressed nor implied, including, but not limited to, the particular purpose. Realtek may make improvements and/or changes in this document or in the product described in this document at any time. This document could include technical inaccuracies or typographical errors.

TRADEMARKS

Realtek is a trademark, of Realtek Semiconductor Corporation. Other names mentioned in this document are trademarks/registered trademarks of their respective owners.

USING THIS DOCUMENT

This document is intended for the software and firmware engineer's general information on the Realtek IP Camera IC. Though every effort has been made to ensure that this document is current and accurate, more information may have become available subsequent to the production of this guide. In that event, please contact your Realtek representative for additional information that may help in the development process.

Product Version

The SDK version corresponding to this document:

SDK Version
SDK v5.3 - SDK v5.3.x

Content

Image

Table

1 Overview

To ensure the security of the RTS3917 Series SoCs, the SDK provides some security features. This guide will introduce two main security features currently supported: Crypto Boot and Secure Boot.

2 Crypto Boot

Crypto Boot refers to the process of encrypting data at rest that is stored on physical devices (such as Flash) to prevent physical access. This process is transparent to upper-layer users, which means that read and write operations will automatically encrypt and decrypt data when the system is running.

Note: It should be noted that Crypto Boot can only protect data at rest. For data in use (such as DDR) and data in transit (such as over a network), users need to employ other methods of protection.

2.1 Encrypting U-Boot Partition

Note: The SDK does not support U-Boot partition encryption.

2.2 Encrypting Kernel Partition

In general, the Kernel partition is relatively small and needs to be loaded into memory all at once. Therefore, the partition can be encrypted using continuous AES encryption with the AES-256-CBC encryption algorithm.

2.3 Encrypting Partitions with Filesystem

For partitions that contain a filesystem, such as Rootfs and User partitions, the loading process is relatively complex and requires consideration of performance issues. To address this situation, the SDK provides two encryption methods: Full-Disk Encryption and File-Based Encryption.

Note: The user partition of the SPI-NOR Flash in the SDK uses the JFFS2 file system, which does not support encryption.

2.3.1 Full-Disk Encryption

Full-Disk Encryption (FDE), also known as Block Device Encryption, uses a single key to encrypt all data on a block device. This means that while the block device is offline, its whole content looks like a large blob of random data, with no way of determining what kind of filesystem and data it contains.

The SDK implements this function using the standard Device Mapper encryption function dm-crypt, which is provided by the Linux kernel. For more information about dm-crypt, please refer to the documentation: dm-crypt (https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/dm-crypt.html).

Other Information

- The SDK supports the squashfs and ext4 file systems.
- When performing Full-Disk Encryption, the SDK uses the AES-256-CBC encryption algorithm and encrypts data multiple times according to the sector size (4096 bytes).
- The initial vector (IV) of each sector is dynamically generated, with its value being the 64-bit little-endian representation of the sector number.

The figure below illustrates the encryption and decryption process based on dm-crypt Fig. 2.1 :

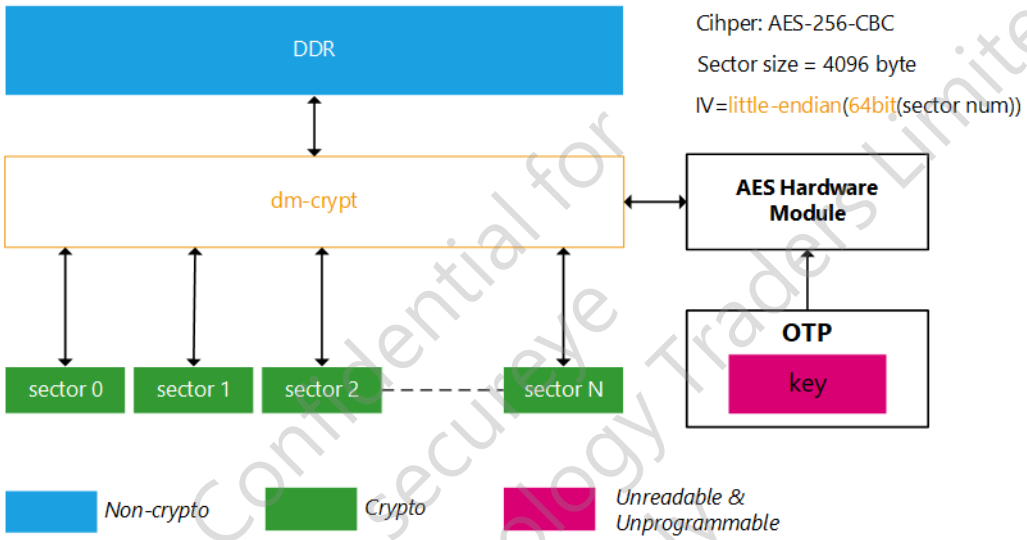

Fig. 2.1: Encryption and decryption based on dm-crypt

2.3.2 File-Based Encryption

File-Based Encryption (FBE), also known as Filesystem-Level Encryption, is a method in which the file system itself encrypts individual files or directories. It allows different files to be encrypted with different keys, but metadata other than the filename cannot be encrypted. This means that other metadata such as file size, number of files, and directory tree layout are still visible.

The SDK implements this function using the fscrypt library, which is provided by the Linux kernel. For more information about fscrypt library, please refer to the documentation: :Filesystem-level encryption (fscrypt) (https://www.kernel.org/doc/html/latest/filesystems/fscrypt.html).

Other Information

- Currently, the SDK only supports the ubifs file system.
- The fscrypt library supports V1 and V2 policies, but due to the limitation of the mkfs.ubifs tool, the current SDK only supports the V1 policies.
- Although it's theoretically possible to provide a separate master key for each file directory, the current SDK only supports providing one master key for one logical partition due to limitations in the mkfs.ubifs tool.

- The SDK supports the Per-file Encryption Keys mechanism: When creating a new encrypted inode, fscrypt generates a 16-byte nonce and uses the Key Derivation Function (KDF) to derive the key from the master key and the nonce.
- The file content is encrypted using the AES-256-XTS algorithm, and the file name is encrypted using the AES-256-CTS-CBC algorithm.

The figure below illustrates the process of file-based encryption Fig. 2.2 :

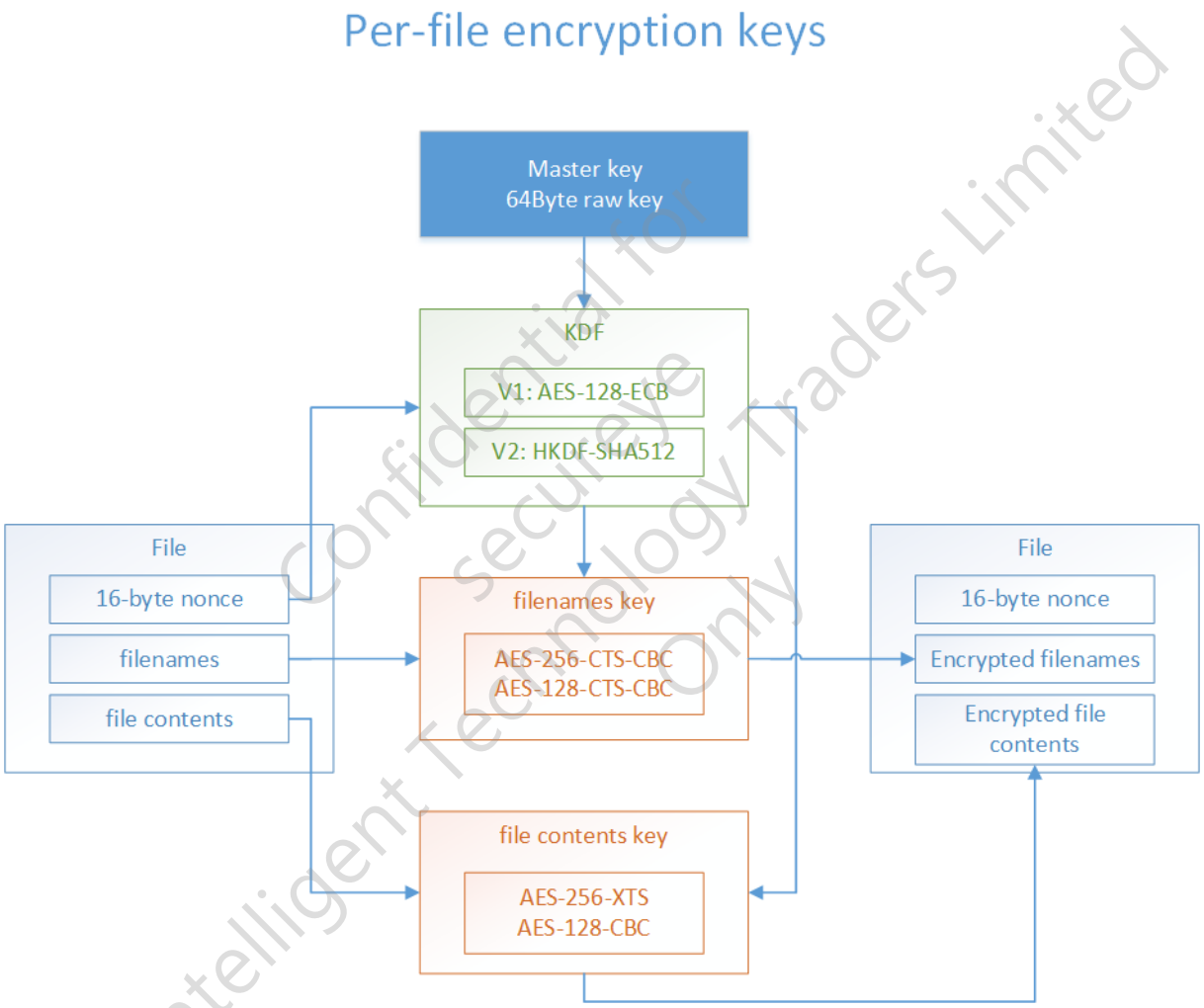

Fig. 2.2: File-Based Encryption

Note: File-Based Encryption is compatible with UBIFS Authentication.

2.3.3 How To Choose

The SDK gives preference to Full Disk Encryption (FDE) and only uses File-Based Encryption (FBE) when the file system does not support FDE, such as in the case of ubifs. The following are the reasons for this approach:

- FDE is generally regarded as having stronger security than FBE because most of FBE's metadata remains unencrypted.
- Keys for FDE can be protected by OTP, but this is not the case for FBE. Please refer to the key protection section for more information.
- dm-crypt requires support for block devices, which means it can only be used with file systems that are based on block devices, such as squashfs and ext4. However, ubifs is based on ubi devices, which operate directly on MTD devices, and therefore dm-crypt cannot be used with ubifs. If ubi must be used, it should be based on ubi block and use the squashfs file system.
- While FBE has flexible configuration options, its configuration can be complex. In contrast, FDE is relatively simple to configure.

The figure below illustrates the file systems supported by each storage device in the SDK Fig. 2.3 :

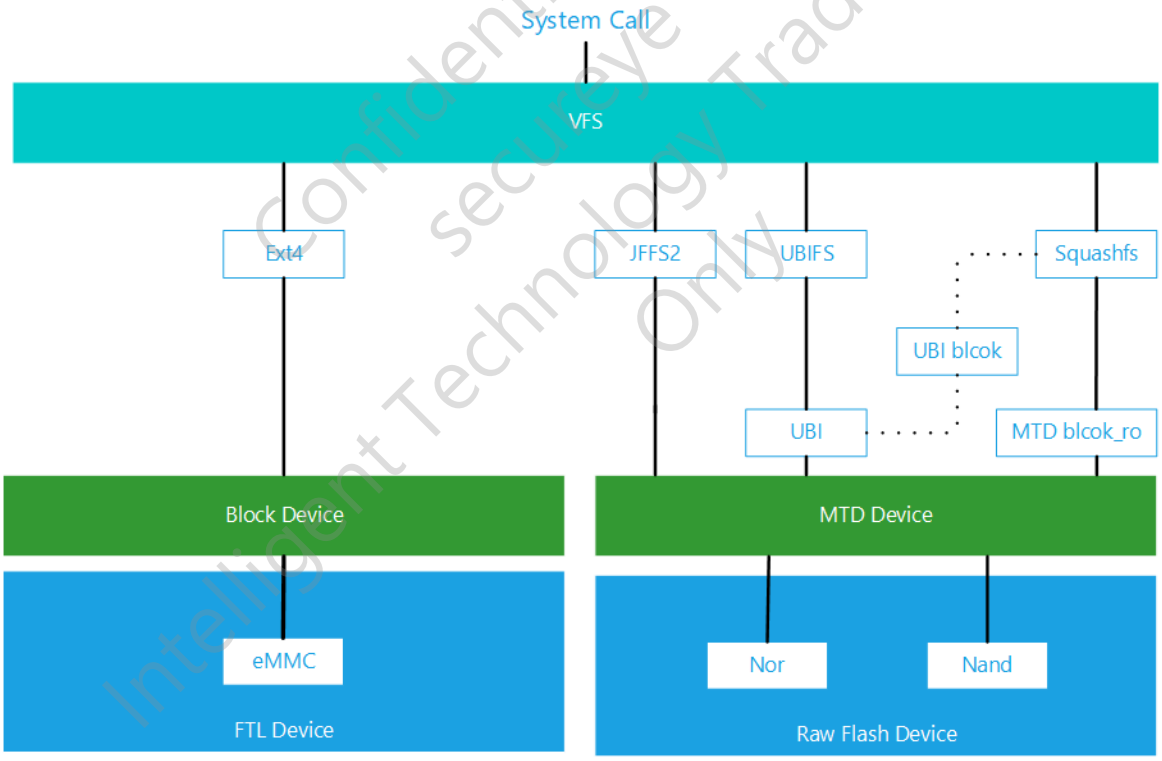

Fig. 2.3: Filesystem in SDK

Note: While the fscrypt library can support both UBIFS and ext4 file systems, the SDK only implements support for UBIFS.

2.4 Key Protection

2.4.1 Kernel Partition

The key is protected by a specific area in the OTP, which cannot be read and changed by the CPU, and can only be accessed by specific hardware encryption modules.

2.4.2 Partitions with Filesystem

Full-Disk Encryption

The key is protected by a specific area in the OTP. By default, the same key is used to encrypt the Kernel partition.

File-Based Encryption

In the Per-file Encryption Keys mechanism, the master key is input data to KDF for processing, so it cannot be protected by OTP. To address this issue, the SDK stores the master key in the initramfs of the Linux kernel and uses the OTP key to protect the kernel. Once the system starts, the Kernel Key Retention Service (Kernel Key Retention Service (https://www.kernel.org/doc/html/latest/security/keys/core.html)) of the Linux kernel is responsible for managing and protecting the master keys to ensure their security.

Note: It should be noted that the key retention service can only ensure that the master key is not accessible in user mode, but an attacker can still potentially obtain the master key from system memory through other means, such as memory dumping. However, this is generally not seen as a problem since the Crypto Boot is primarily designed to prevent offline attacks, during which the key is not present.

---

3 Secure Boot

Unlike Crypto Boot, the primary purpose of secure boot is to prevent the software running on the device from being tampered with. If the software is tampered with, the device may not boot properly. The software implementation of secure boot includes the CPU BootRom, bootloader, Linux kernel, and other partitions with file systems (such as rootfs and user partitions). The entire secure boot process is called the Chain of Trust (CoT). The front-level components are responsible for verifying the latter-level components, and only successful level-by-level verification can ensure that the device can be safely started and enter a trusted working state.

The structure of the secure boot is as below Fig. 3.1 :

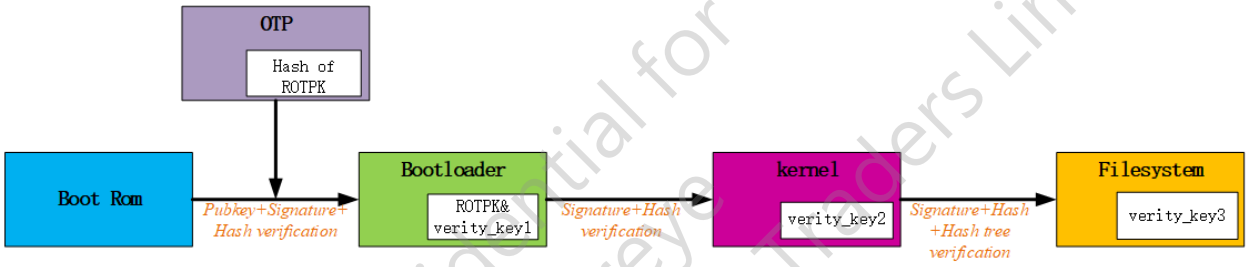

Fig. 3.1: structure of the secure boot

Note: During the secure boot process, RSA-2048 is the asymmetric encryption algorithm used, and the certificate format used is X509.

3.1 Verify U-Boot Partition

The implementation is based on ATF-TBB (ARM Trusted Firmware - Trusted Board Boot). Due to hardware limitations, only the process of booting from BL1 (ROM code) and BL1 verifying BL2 (U-Boot) is implemented.

3.1.1 Verification Process

The entire verification process starts with the Rom Code, which is considered the root of trust in the Chain of Trust (CoT), as it cannot be tampered with.

The verification process is divided into two steps:

1. Verify certificate:

- Read the ROTPK from the BL2 (U-Boot) content certificate;
- Validate the “Sign value” of the certificate using the read ROTPK, if the verification succeeds, the certificate is passed.

2. Verify Root of Trust Public Key(ROTPK):

- Read the ROTPK from the BL2 (U-Boot) content certificate, calculate the SHA256 value of the ROTPK, denoted as H1;
- Read the “Hash of ROTPK” from OTP, denoted as H2;
- Compare H1 and H2, if they are equal, the ROTPK verification is passed.

2. Verify U-Boot Partition:

- Calculate the SHA256 value of the uboot.bin in BL2, denoted as H3;
- Obtain the “Hash of uboot.bin” from the BL2 content certificate, denoted as H4;
- Compare H3 and H4, if they are equal, the U-Boot partition verification is passed.

Finally, load the boot image file fip.bin. The structure of “BootRom Verification U-Boot” is illustrated in the figure below. Fig. 3.2 :

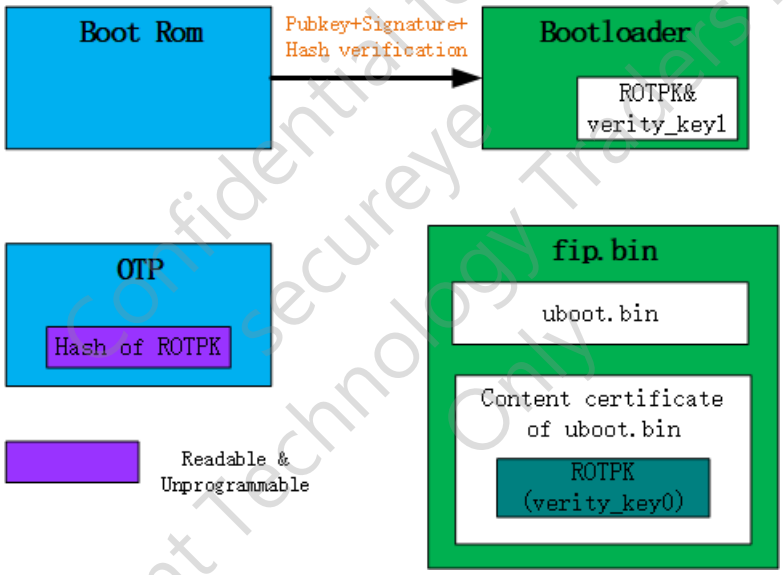

Fig. 3.2: BootRom verify U-Boot

3.2 Verifying Kernel Partition

U-Boot 2013.07 introduced a new feature called “Verified Boot”, which allows verification of the kernel and other images while still allowing for field upgrades.

3.2.1 Verification Process

The public key verity_key1 is stored in the U-Boot partition and has already been verified by the Rom Code.

Verify Kernel Partition:

- Calculate the SHA256 hash value of vmlinuz.img, denoted as H1;
- Decrypt the “Kernel Sign Value” with the public key verity_key1, denoted as H2;

- Compare H1 and H2, if they are equal, the Kernel verification is passed.

Load the Kernel image. The structure of "U-Boot Verify Kernel" is illustrated in the figure below Fig. 3.3 :

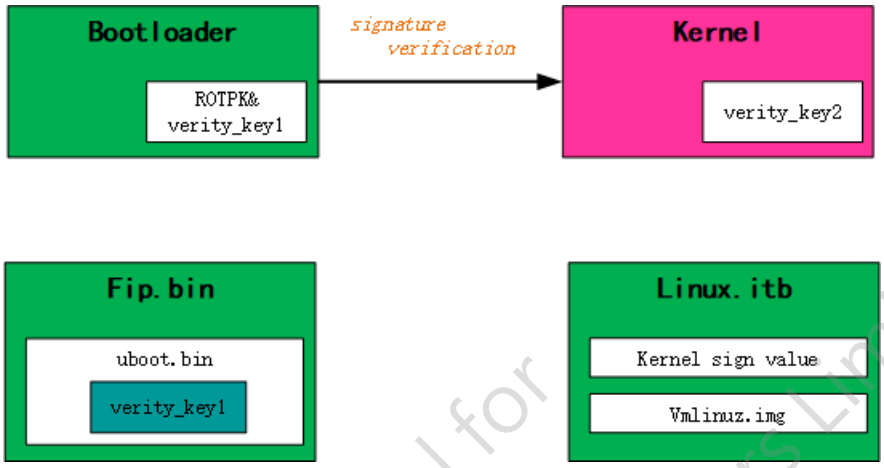

Fig. 3.3: U-Boot Verify Kernel

3.3 Verifying Partitions with Filesystem

Similar to the situation described in Crypto Boot, for partitions with file systems, the SDK also needs to provide different implementations for different situations.

- The SDK uses standard Device Mapper related verification functions provided by the Linux kernel (such as dm-verity and dm-integrity) to verify file systems based on block devices.
- The SDK uses the UBIFS Authentication function provided by the Linux kernel to verify the ubifs file system.

Note: In the SDK, the user partition of SPI-NOR Flash uses the JFFS2 file system, which does not support authentication.

3.3.1 dm-verity

The SDK uses the standard Device Mapper verification function dm-verity provided by the Linux kernel to perform transparent read-only integrity checks on block devices. For more information about dm-verity, please refer to the document: dm-verity (https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/verity.html).

Working Principle

- The SDK calculates the hash value for each block of the block device and generates a Hash Tree using these hash values as leaf nodes. Any attempt to modify a data block alters its parent node's hash value, resulting in a different root node hash value (Root Hash) from the original data.
- To prevent tampering with the hash tree, the Root Hash must be secured. The signature value of the dm-verity mapping table (including the Root Hash) can be stored in the verity metadata. During startup, the signature of the dm-verity mapping table can be verified using the pre-stored public key.

- When handling a read request, dm-verity reads the relevant data block, calculates its hash value, retrieves the corresponding hash value from the hash tree, and verifies whether the two hash values match.

The tree looks something like: Fig. 3.4 :

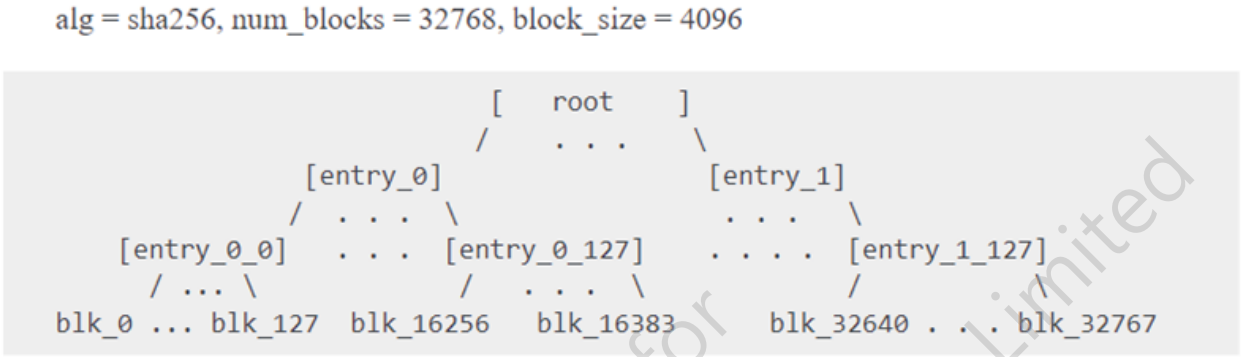

Fig. 3.4: dm-verity HashTree

Verify Rootfs Partition

The Rootfs partition is read-only and can be verified through dm-verity. The public key for verification, verity_key2, is stored in the Kernel partition and has been verified by U-Boot.

Verify Root Hash:

- The Kernel mounts the initramfs and runs the init script.
- The system parses the file system information (squashfs/ext4) and retrieves the verity metadata attached to the end of the file system.
- The system verifies the signature of the dm-verity mapping table, including the Root Hash of the hash tree, stored in the verity metadata using the public key verity_key2.

Mount Rootfs Partition:

- Create a mapping between a virtual device (dm-verity) and a physical device.
- Access the physical device through dm-verity to realize automatic verification when reading.
- Execute the *switch_root* command to mount the Rootfs.

The structure of “Kernel Verify Rootfs” is illustrated in the figure below Fig. 3.5 :

Verify User Partition

By default, the SDK sets one read-only partition called “rodata”. Additionally, the public key “verity_key2” is used by default, which is stored in the Rootfs partition and has already been verified by the Kernel.

Once the Rootfs partition is mounted, execute the “etc/init.d/S02fsmgr” script to initiate the verification and mounting process. This process is essentially the same as the verification process for the Rootfs partition.

The structure of “Rootfs Verify rodata” is illustrated in the figure below Fig. 3.6 :

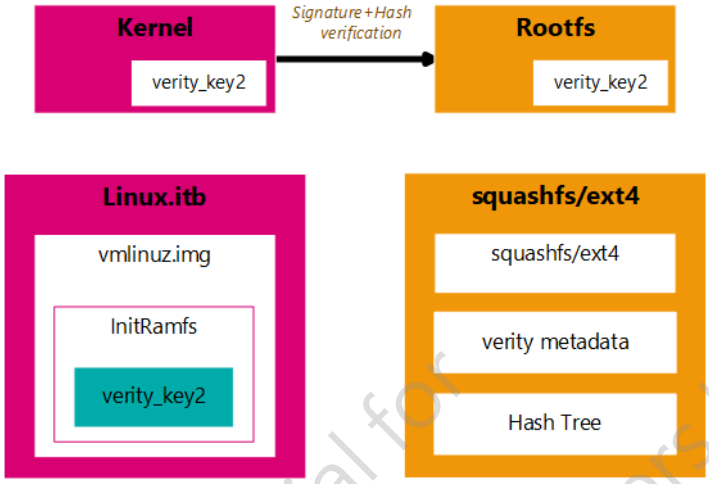

Fig. 3.5: Kernel Verify Rootfs

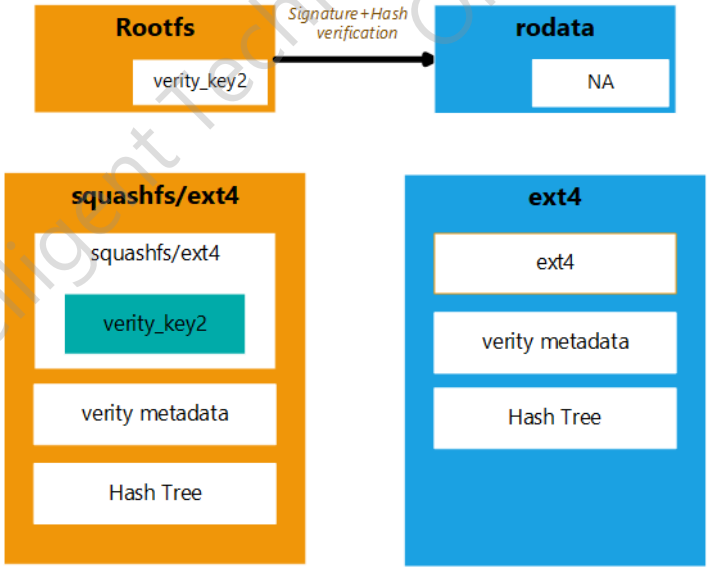

Fig. 3.6: Rootfs Verify rodata

3.3.2 dm-integrity

The SDK uses the standard Device Mapper verification function dm-integrity provided by the Linux kernel to perform transparent read-write integrity checks on block devices. For more information about dm-integrity, please refer to the document: dm-integrity (https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/dm-integrity.html).

Working Principle

- The dm-integrity target emulates a block device that has additional per-sector tags that can be used for storing integrity information.
- To guarantee write atomicity, the dm-integrity target uses journal, it writes sector data and integrity tags into a journal, commits the journal and then copies the data and integrity tags to their respective location.
- Currently, dm-integrity's standalone mode is used ,it will automatically generate and verify the integrity tags. The CRC32 algorithm is used by default.

Verify User Partition

In the SDK, the read-write partition named "userdata" can be verified using dm-integrity.

Once the Rootfs partition is mounted, execute the "etc/init.d/S02fsmgr" script to initiate the verification and mounting process.

Mount userdata Partition:

- If it is the first time starting up, the SDK will automatically format the userdata partition. Otherwise, the SDK will proceed to the next step.
- Create a mapping between a virtual device (dm-integrity) and a physical device.
- Access the physical device through dm-integrity to realize automatic verification when reading and writing.

Warning:

- If a power cut or other unforeseen circumstances occur during the formatting of the userdata partition, it may result in partition corruption.
- To avoid this, the user manually enables the BR2_CONFIG_SECURE_BOOT_PACK_DMINTEGRITY, at compile time, the SDK automatically formats user data partitions. Note that this option grants SDK with root privileges at compile time, and the final usable partition size will be limited by the current partition file size.
- The current implementation provides protection against accidental modifications, but it may not provide complete protection against malicious modifications.

3.3.3 UBIFS Authentication

UBIFS Authentication enables UBIFS to verify the authenticity and integrity of metadata and file content stored on flash memory. For more information about UBIFS Authentication, please refer to the document: UBIFS Authentication Support (https://www.kernel.org/doc/html/latest/filesystems/ubifs-authentication.html#ubifs-authentication).

Working Principle

The HMAC algorithm is mainly used to verify the authenticity and integrity of each data structure, including the Index, Journal, and LPT (LEB property tree) in UBIFS. For more detailed information, please refer to the kernel document "UBIFS Authentication Support".

Key Management

To simplify the process, the same key is used to compute HMAC for each data structure. The key is secured and managed by the Kernel Key Retention Service in Linux (Kernel Key Retention Service (https://www.kernel.org/doc/html/latest/security/keys/core.html)) to ensure its security.

Offline Signed Image

When actually making an image, use PKCS#7 digital signature instead of HMAC to protect data. The specific implementation steps are as follows:

- Calculate the Hash of Master Node and store it in SuperBlock.
- Sign the SuperBlock Node and append the signed value to the SuperBlock Node.
- Recalculate the HMAC of SuperBlock Node and Master Node during mount, and switch to HMAC authentication mode. At this point the signature value is useless.

Note:

- UBIFS Authentication is compatible with File-Based Encryption.
- The tail of the journal which may not have a authentication node (for example, because of a power cut) cannot be authenticated and is skipped during journal replay.

---

4 Use Process

4.1 Compiling the Image

4.1.1 Compilation Commands

The *make secure_boot* series of commands implement integrated compilation for Secure and Crypto Boot modes. The specific descriptions of the commands are as follows:

```
$ secure_boot [M=0|1|2]  - make world, and enable crypto or secure boot
                           0: enable crypto Boot
                           1: enable secure boot
                           2: enable encrypted and secure boot. default value
$ secure_clean           - Disable crypto Boot and secure boot
```

Note: When switching between modes, if you want to get a clean compilation environment, you need to relaunch and execute *make rp*, then execute the required compilation command.

4.1.2 Key Configuration

Usually, manual configuration is not necessary, and it will be automatically configured when executing the *make secure_boot* command series. If you need to make some changes to the name and save location of the keys, you can open the Buildroot configuration image interface for configuration by executing *make menuconfig* in the compilation workspace.

```
$ make menuconfig

External options  --->
    System packages  --->
            -*- rtskey  --->
            ($(BASE_DIR)/rtskey) rtskey output directory
            (/etc/keys) keys target directory
            (crypto_key) crypto boot key name
            (crypto_iv) crypto boot iv name
            (verity_key0) root key name
            (verity_key1) kernel key name
            (verity_key2) data key name
            (fbe_key) FBE key name
            (ubifs_auth_key) ubifs authtication key name
```

4.1.3 User Partition Configuration

By default, Crypto Boot is enabled for the user partition, but Secure Boot is disabled. This is because:

- User partition is generally writable, but enabling Secure Boot will result in poor write performance.
- For writable partitions, the UBIFS authentication feature may cause the tail data of the journal to be lost in the event of anomalies such as power cut. Please refer to *UBIFS Authentication* .
- There are some limitations when using dm-integrity for verification on writable partitions. Please refer to *dm-integrity*.

```
$ make menuconfig

External options  --->
    [*] Support crypto boot
        [*]   enable user partition
    [*] Support secure boot
        [ ]   enable user partition
```

4.1.4 Key Generation

After executing the command, the SDK will create an “rtskey” directory in the compilation workspace, and generate the necessary keys for the current mode in it. The contents of the directory are shown in the following table: *rtskey file description*

Table 4.1: rtskey file description

File Name	Description
crypto_key.bin	Encryption key, need to write OTP
crypto_iv.bin	Encryption IV of Kernel, need to write OTP
fbe_key.bin	FBE key (must be 64Byte)
verity_key0.key	The private key in PEM format can generate the corresponding public key from the private key, which is used for BootRom to verify U-Boot
verity_key1.key	The private key in PEM format can generate the corresponding public key from the private key, which is used for U-boot to verify kernel
verity_key2.key	The private key in PEM format can generate the corresponding public key from the private key, which is used for kernel to verify partitions with filesystem
verity_key1.crt	Self-signed public key certificate, dynamically generated at compile
verity_key2.crt	Self-signed public key certificate, dynamically generated at compile
verity_key0_pub.der.sha256.bin	Hash of public key,need to write OTP, dynamically generated at compile
ubifs_auth_key.bin	Keys required for HMAC in UBIFS Authentication

Note: The SDK will not automatically overwrite existing keys.

Warning: The contents of this directory must be copied out of the workspace and kept properly.

You can also manually generate the key through openssl, for example:

```
$openssl enc -aes-256-cbc -k secret -P | awk -Fkey= '{printf $2}' | xxd -r -ps >␣
↪crypto_key.bin
$openssl enc -aes-256-cbc -k secret -P | awk -F'iv =' '{printf $2}' | xxd -r -ps >␣
↪crypto_iv.bin

$openssl genrsa -out verity_key0.key 2048
$openssl genrsa -out verity_key1.key 2048
$openssl genrsa -out verity_key2.key 2048
```

Then overwrite the file with the same name in the rtskey directory, and it will take effect when recompiling.

4.1.5 Image Generation

The *make pack* command is supported, which will pack the partition specified in merge.ini and generate the image file linux.bin, which is located in the images directory under the working directory.

Note: The merge.ini file is located in the working directory of the SDK.

4.2 Burn Image

Note: Before burning the image, the keys must be written to the OTP of the device in normal mode. For specific operation methods, please refer to *OTP Programming* .

Usually, the *update all* command is used to update image. For more details, please refer to the "Update Image in U-Boot" section in the "SDK User Guide". If you need to burn individual partitions separately, please refer to the table below.

Update Crypto Boot image, see table *Update Crypto Boot image*

Table 4.2: Update Crypto Boot image

Partition	SPI-NOR Flash	SPI-NAND Flash	Update Command
uboot	uboot.bin	uboot.bin	update uboot
kernel	uImage.crypted	uImage.crypted	update kernel
rootfs	rootfs.squashfs.crypted	rootfs.ubi	update rootfs

Update Secure Boot Image, see table *Update Secure Boot Image*

Table 4.3: Update Secure Boot Image

Partition	SPI-NOR Flash	SPI-NAND Flash	Update Command
uboot	fip.bin	fip.bin	update uboot
kernel	linux.itb	linux.itb	update kernel
rootfs	rootfs.squashfs.signed	rootfs.ubi	update rootfs

Update the Secure and Crypto Boot Image, see table *Update the Secure and Crypto Boot Image*

Table 4.4: Update the Secure and Crypto Boot Image

Partition	SPI-NOR Flash	SPI-NAND Flash	Update Command
uboot	fip.bin	fip.bin	update uboot
kernel	linux.crypted.itb	linux.crypted.itb	update kernel
rootfs	rootfs.squashfs.signed.crypted	rootfs.ubi	update rootfs

4.3 Configure Work Mode

Configuring the Work Mode is required to enable Secure and Crypto Boot. For information on Work Mode, please refer to the “Work Mode” chapter in the “SDK User Guide”.

- Enabling Crypto Boot does not require additional setting of the Work Mode, just keep the default “SPI-NOR/SPI-NAND Boot Mode”.
- After enabling Secure Boot, it is necessary to configure as “ROM Secure Boot Mode” by setting GPIO[7-4] to 4’b0010

5 OTP Programming

OTP (One Time Programmable) is a non-volatile memory technology that allows for programming only once. Once the memory is programmed, it cannot be changed or reprogrammed, but it retains its value upon loss of power. OTP memory is commonly used to store critical data, such as security keys or system configuration information to prevent unauthorized access or tampering.

The default value of each bit in RTS3917/RTS3918 OTP is 1 and can be modified to 0. The entire OTP is divided into 57 groups, as follows:

- Group 0 consists of 128 bits, each with a specific function.
- Groups 1 to 56 each contain 288 bits. Among them, the lower 256 bits are the data segment, and the upper 32 bits are the ECC bits..

The allocation of groups is shown in the table below *Group distribution*

Table 5.1: Group distribution

Group	Function
Group0	Function BIT, see the table below for details
Group1 - 4	4 sets of keys for AES hardware modules
Group5 - 6	2 groups store the SHA256 value of the RSA public key
Group7 - 10	4 groups are used to store IV
Group11	Ethernet Mac
Group12 - 56	Dummy Group

The function description of each bit of Group0 is shown in the table below *Function description of each bit of Group0*

Table 5.2: Function description of each bit of Group0

Group	BIT	Description
AES Group	0 - 3	Control programming and reading permissions of Group1 to Group4: 1: Group programmable, CPU readable 0: Group is not programmable, CPU is not readable
RSA Group	4 - 5	Control programming and reading permissions of Group5 to Group6: 1: Group programmable, CPU readable 0: Group is not programmable, CPU readable
IV Group	6 - 9	Control programming and reading permissions of Group7 to Group10: 1: Group programmable, CPU readable 0: Group is not programmable, CPU readable
Other Groups	10 - 56	Control programming and reading permissions of Group11 to Group57: 1: Group programmable, CPU readable 0: Group is not programmable, CPU readable
IC info sign	95	Crypto Boot Flag: 1: Read the key from the SD card 0: Read the key from OTP
	97	Secure Boot Flag: 1: Non-Secure Boot 0: Secure Boot

5.1 Preparation

Burning key into OTP requires the support of kernel driver and application layyer tools. All the burning operations are under in normal boot mode of the device!

1. Make sure the kernel contains OTP driver

Open the Linux kernel configuration interface by running *make linux-menuconfig* in the compilation workspace, and select the following options:

```
$ make linux-menuconfig

=> Device Drivers
    => NVMEM Support
        [*] /sys/bus/nvmem/devices/*/nvmem (sysfs interface)
        <*> Realtek OTP Support
        [*]   Realtek OTP Writable
```

2. Prepare the OTP burning tool

Open the Buildroot configuration interface by running *make menuconfig* in the compilation workspace, and select the following options:

```
$ make menuconfig

=> External options
    => System packages
        [*] otp_mfg
```

5.2 Starting Programming

Warning: Once each bit is burned into the OTP, it cannot be changed. Therefore, great care must be taken when burning the OTP.

5.2.1 Crypto Boot OTP Programming

To enable Crypto Boot mode, the AES 256-bit key should be burned into Group1 of the OTP, and the IV should be burned into Group7 of the OTP.

1. Read the contents of crypto_key.bin (aes-key) and crypto_iv.bin (aes-iv) on the PC:

```
$ cat rtskey/crypto_key.bin | xxd -ps -c 32
b854ad1716c4f10e24a022c8752d167cca619f91ec908d9d3ab39b4fe545c905    // aes-
↪key
$ cat rtskey/crypto_iv.bin | xxd -ps
62e6f2984486d8f7ea059aca8b67af3b                                    // aes-iv
```

2. Burn the content onto the corresponding group of the OTP using the otp_mfg tool on the device:

```
$ otp_mfg -w --ecc [aes-key] 16     // 16 represents the offset byte, --ecc␣
↪represents the tool
                                    // will calculate the ECC value and␣
↪write it into Group
$ otp_mfg -w --ecc [aes-iv] 208
$ otp_mfg -r --pg                   // Check whether the write is correct
$ otp_mfg -w --bit 0 95             // Flag bit
$ otp_mfg -w --bit 0 0              // Control bit
$ otp_mfg -w --bit 0 6              // Control bit
```

5.2.2 Secure Boot OTP Programming

To enable Secure Boot mode, you need to burn the SHA256 value of ROTPK into Group5 of the OTP.

1. Read the contents of verity_key0_pub.der.sha256.bin on the PC:

```
$ cat rtskey/verity_key0_pub.der.sha256.bin | xxd -p -c 32
  d22957af9d0654879c7d6d3a8b5394994682db817453612f09927d1c97784eeb // hash
```

2. Burn the content onto the corresponding group of the OTP using the otp_mfg tool on the device:

```
$ otp_mfg -w --ecc [hash] 144      // 144 represents the offset byte, --ecc␣
↪represents the tool
                                   // will calculate the ECC value and write␣
↪it into Group
$ otp_mfg -r -pg                   // Check whether the write is correct
$ otp_mfg -w --bit 0 97            // Flag bit
$ otp_mfg -w --bit 0 4             // Control bit
```

5.2.3 Secure & Crypto Boot OTP Programming

If both Secure and Crypto Boot modes are turned on, the above two burning steps need to be performed.

5.3 Checking Results

Once the data is written into OTP, it cannot be changed again. In particular, Group1 where the AES key is located, after completing the writing and setting the control bit to 0, the tool will not be able to read the actual value of this Group, and it will always display as all 0s. Therefore, users need to properly store all the written keys.

otp_mfg -r -pg is a command to check if the data is correctly written into the OTP:

```
# otp_mfg -r -pg
group000-000(0000): ae ff ff ff ff ff ff ff ff ff ff 7f fd ff ff ff
group001-002(0016): 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  //
↪ aes-key
group003-004(0052): ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff
group005-006(0088): ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff
group007-008(0124): ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff
group009-010(0160): d2 29 57 af 9d 06 54 87 9c 7d 6d 3a 8b 53 94 99 46 82 db 81 74  //
↪ Hash of ROTPK
group011-012(0196): ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff
group013-014(0232): 62 e6 f2 98 44 86 d8 f7 ea 05 9a ca 8b 67 af 3b ff ff ff ff ff  //
↪ aes-iv
```

5.4 Updating the Image

After OTP burning, update the image file of Secure and Crypto Boot. Please refer to *Burn Image* .

6 Functional Verification

6.1 Verifying Crypto Boot

1. The image with the otp_mfg tool needs to be burned in advance on the device. This tool is used to write the data related to crypto boot or secure boot into the hardware OTP, for the configuration of this tool, please refer to *Preparation* .

After the configuration is completed, download the image to the device. After entering the system, you can find the otp_mfg tool in the /bin directory.

```
$ls /bin/otp_mfg
/bin/otp_mfg
```

Note: At this point, the image with the otp_mfg tool should be the image without the crypto boot function/secure boot function, you can refer to the "RTS3917 Series SDK Development User Guide" to create this image

2. Compile the image with the crypto boot function enabled and write the corresponding aes-key and aes-iv into the OTP.

(1)To ensure a clean development environment, directly re-run the launch.sh script file under the SDK directory, select the configuration file of 3917n, and then execute the command "make rp" in the generated workspace to rebuild the platform.

Note: In this example, 3917n is selected for testing. The workspace is generated after executing the launch.sh script and is by default located in out/rts3917n_base under the SDK directory

(2)After entering the workspace, execute the following command:

```
make secure_boot M=0
```

Please refer to the specific meaning of the command *Compilation Commands* .

After the command is executed successfully, two files, crypto_iv.bin and crypto_key.bin, will be generated in the rtskey folder under the SDK directory.

(3)Burn the AES 256-bit key to Group1 of OTP and burn the IV to Group7 of OTP

①Read crypto_key.bin(aes-key) and crypto_iv.bin(aes-iv) respectively on the PC:

```
$cat rtskey/crypto_key.bin | xxd -ps -c 32
8f1324ed2a225c621e5bc757eac706b3d1b822981363a65e7d2a75b68e3ab766    // aes-
↪key
$cat rtskey/crypto_iv.bin | xxd -ps
c20a7511058488a93d0d439ef97e09e7   // aes-iv
```

Note: During the actual usage process, please use the content actually read as aes-key and aes-iv respectively

②On the device, the aes-key and aes-iv are burned into the corresponding groups through the tool otp_mfg:

```
$otp_mfg -w --ecc [aes-key] 16  // 16 represents the offset byte, and --ecc␣
↪indicates that the ECC value is calculated by the tool and written into␣
↪the Group
$otp_mfg -w --ecc [aes-iv] 208
$otp_mfg -r --pg                // Check if the write is correct. Please␣
↪carefully verify whether the displayed content is consistent with what is␣
↪read on the PC
$otp_mfg -w --bit 0 95          // Flag bit
$otp_mfg -w --bit 0 0           // Control bit
$otp_mfg -w --bit 0 6           // Control bit
```

Note: Please replace [aes-key] and [aes-iv] in the command of step ② respectively with the aes-key and aes-iv read in step ①, the symbols "[" and"] "should be removed

Warning: If it is the first time to write aes-key and aes-iv and the corresponding flag bit and control bit are not set, at this time, using the command otp_mfg -r –pg, the aes-key and aes-iv written to the OTP can be seen. If it is not the first write and the corresponding flag bit and control bit are set, the positions corresponding to AES-key are all 0. Because for Group1 where the AES key is located, after completing the write and setting the control position to 0, the otp_mfg tool will be unable to read the actual value of this Group, it will be fixedly displayed as all zeros.

3. Update the image with encrypted startup function to the device.

(1)It is necessary to check in the workspace whether the merge.ini file contains uboot, kernel, and rootfs. If the corresponding components are commented out, they need to be uncommented.

(2)Execute the command "make pack" in the workspace to package the image and generate the linux.bin file. The image file used for packaging is located in the "images" directory of the workspace, and the corresponding file is:

```
uboot image: u-boot.bin
kernel image: uImage.crypted
rootfs image: rootfs.squashfs.crypted
```

(3)During the uboot command line stage of the device, use Ethernet and the command "update all" to download the linux.bin file to the device, For this step, please refer to the "Burning the Image via TFTP" section in the "U-boot" section of the "RTS3917 Series SDK Development User Guide".

(4)After the device restarts, enter the system and check whether the dm-crypt-newroot file is generated in the /dev/mapper directory. If it is generated, it indicates that the encryption startup is successful.

6.2 Verifying Secure Boot

The verification process of this function is basically the same as that of the crypto boot verification process, except that the corresponding files generated are different. After downloading the image with the secure boot function to the device, it is necessary to set the working mode of the device to the secure mode in order to start the system smoothly.

1. The image with the otp_mfg tool needs to be burned in advance on the device. This tool is used to write the data related to crypto boot or secure boot into the hardware OTP, for the configuration of this tool, please refer to *Preparation* .

After the configuration is completed, download the image to the device. After entering the system, you can find the otp_mfg tool in the /bin directory.

```
$ls /bin/otp_mfg
/bin/otp_mfg
```

Note: At this point, the image with the otp_mfg tool should be the image without the crypto boot function/secure boot function, you can refer to the "RTS3917 Series SDK Development User Guide" to create this image

1. Compile the image with the secure boot function enabled and burn the SHA256 value of ROTPK to Group5 of OTP.

(1)To ensure a clean development environment, directly re-run the launch.sh script file under the SDK directory, select the configuration file of 3917n, and then execute the command "make rp" in the generated workspace to rebuild the platform.

Note: In this example, 3917n is selected for testing. The workspace is secure_boot after executing the launch.sh script and is by default located in out/rts3917n_base under the SDK directory

(2)After entering the workspace, execute the following command:

```
make secure_boot M=1
```

Please refer to the specific meaning of the command *Compilation Commands* .

After the command is executed successfully, the following file will be generated in the rtskey folder under the SDK directory:

```
verity_key0.key                 //The private key in PEM format can generate␣
↪the corresponding public key from the private key, which is used for␣
↪BootRom to verify U-Boot
verity_key0_pub.der.sha256.bin  //Hash of public key,need to write OTP,␣
↪dynamically generated at compile
verity_key1.crt                 //Self-signed public key certificate,␣
↪dynamically generated at compile
verity_key1.key                 //The private key in PEM format can generate␣
↪the corresponding public key from the private key, which is used for U-
↪boot to verify kernel
```

(continues on next page)

(continued from previous page)

```
verity_key2.crt                 //Self-signed public key certificate,␣
↪dynamically generated at compile
verity_key2.key                 //The private key in PEM format can generate␣
↪the corresponding public key from the private key, which is used for␣
↪kernel to verify partitions with filesystem
```

(3)Burn the SHA256 value of ROTPK to Group5 of OTP.

①To read the content of verity_key0_pub.der.sha256.bin on a PC, use the following command:

```
$cat rtskey/verity_key0_pub.der.sha256.bin | xxd -p -c 32
232e21b095f79cf8e2eeb9626acb614c3b34728ad8ac5620787bc6607744d30a  //
↪hash
```

Note: In actual use, please take the content actually read as the Hash

②On the device, burn the hash to the corresponding Group through the tool otp_mfg:

```
$otp_mfg -w --ecc [hash] 144 // 144 represents the offset byte, and -
↪-ecc indicates that the ECC value is calculated by the tool and␣
↪written into the Group
$otp_mfg -r -pg // Check if the write is correct.
$otp_mfg -w --bit 0 97 // Flag bit
$otp_mfg -w --bit 0 4 // Control bit
```

Note: Please replace the hash content in the command of step ② with the corresponding hash obtained in step ①, and also remove the "[" "]"

Warning: When executing the command otp_mfg -r -pg, please carefully check the written hashh

3. Update the image with the secure boot function to the device.

(1)It is necessary to check in the workspace whether the merge.ini file contains uboot, kernel, and rootfs. If the corresponding components are commented out, they need to be uncommented.

(2)Execute the command "make pack" in the workspace to package the image and generate the linux.bin file. The image file used for packaging is located in the "images" directory of the workspace, and the corresponding file is:

```
uboot image: fip.bin
kernel image: linux.itb
rootfs image: rootfs.squashfs.signed
```

(3)During the uboot command line stage of the device, use Ethernet and the command "update all" to download the linux.bin file to the device, For this step, please refer to the "Burning the Image via TFTP" section in the "U-boot" section of the "RTS3917 Series SDK Development User Guide".

(4)Before restarting the device, it is necessary to cut off the power supply to the device first, and then set the working mode of the device to safe mode. For the safe mode of 3917n, GPIO7 to GPIO4 should be set to 0010 in sequence. After the Settings are completed, the device should be powered on and started again.

Note: For the working mode, you can refer to the "Working Mode" section of the "RTS3917_Series_Emergency_Mode_User_Guide_Manual"

Warning: If the working mode of the device is not set to safe mode, the device cannot start up normally

(5)Enter the system and check whether the dm-verity-newroot file is generated under the /dev/mapper directory. Moreover, secure boot also has a similar log as follows:

①The log of ATF starting u-boot is as follows:

```
NOTICE:  Booting Trusted Firmware
NOTICE:  BL1: v2.3():V2.0_TO-3-g4ae38097
NOTICE:  BL1: Built : 13:39:47, Mar  7 2022
INFO:    BL1: RAM 0x19004000 - 0x1900f000
INFO:    Using crypto library 'mbed TLS'
INFO:    BL1: Loading BL2
INFO:    Using FIP in nor flash mode
INFO:    Loading image id=6 at address 0x80040000
INFO:    nor flash security mode
INFO:    Image id=6 loaded: 0x404551c - 0x404598b
INFO:    Using FIP in nor flash mode
INFO:    Loading image id=1 at address 0x404551c
INFO:    nor flash security mode
INFO:    Image id=1 loaded: 0x4000088 - 0x404551c
NOTICE:  BL1: Booting BL2
INFO:    Entry point address = 0x4000088
INFO:    SPSR = 0x1d3
```

②The log of u-boot starting the kernel is as follows:

```
## Loading kernel from FIT Image at 80400000 ...
   Using 'config@1' configuration
   Verifying Hash Integrity ... OK
   Trying 'kernel@1' kernel subimage
     Description:  Linux kernel
     Created:      2024-06-05  10:17:49 UTC
     Type:         Kernel Image
     Compression:  uncompressed
     Data Start:   0x804000c0
     Data Size:    3339928 Bytes = 3.2 MiB
     Architecture: ARM
     OS:           Linux
     Load Address: 0x81000000
     Entry Point:  0x81000000
     Sign algo:    sha256,rsa2048:verity_key1
     Sign value:  ␣
↪0d0c87dc171bc554139b5d0ea85d47a0f6b89fcd0a42ee8ca24ff699d0
                  ␣
↪af67a63a7546abe8eebf37e0feb33a4cfe4fceb572a256d0f1aac21b28
                  ␣
↪d8f65d5b55ad0f8ea91e6a86411c2ddb904c236298a2f89806d027a633
                  ␣
```

(continues on next page)

(continued from previous page)

```
↪acd1bc7d723bf28369439f8b8f9aac65b0df4521ed94da80af497dae5d
                    ␣
↪cbe7a670c14fc537f61626d4dfd4841d79641e7bf490de6b8074e2e5a1
                    ␣
↪388f465c2b4edabece65f632e1e4da689b9ad27db4335d76f8e7f0bff7
                    ␣
↪77c9c5b86cdc779345567b72488474fa4403e8484fa0d7e9ec7f222ba8
                    ␣
↪325bb6b6426c133cfe4a63ed524758304d1b80b2ccbc0ccedd7683c2bf
                     80d7bad513b122c654b878eaf4f62befc29f429584875b95
     Timestamp:    2024-06-05  10:17:49 UTC
   Verifying Hash Integrity ... sha256,rsa2048:verity_key1+ OK
## Flattened Device Tree blob at 8774ff34
   Booting using the fdt blob at 0x8774ff34
   Loading Kernel Image
```

If all the above phenomena occur, it indicates that the verification of the safe start function has been successful.

6.3 Verifying Secure & Crypto Boot

This functional verification process integrates the encrypted startup verification process and the secure startup verification process. Moreover, since the secure startup function is enabled, after downloading the image with the secure encrypted startup function to the device, the device also needs to set the working mode to the safe mode after power failure, and then restart.

1. The image with the otp_mfg tool needs to be burned in advance on the device. This tool is used to write the data related to crypto boot or secure boot into the hardware OTP, for the configuration of this tool, please refer to *Preparation* .

After the configuration is completed, download the image to the device. After entering the system, you can find the otp_mfg tool in the /bin directory.

```
$ls /bin/otp_mfg
/bin/otp_mfg
```

Note: At this point, the image with the otp_mfg tool should be the image without the crypto boot function/secure boot function, you can refer to the "RTS3917 Series SDK Development User Guide" to create this image

2.Compile the image with the secure encryption boot function enabled, burn the AES 256-bit key to Group1 of OTP, and burn the IV to Group7 of OTP. Burn the SHA256 value of ROTPK to Group5 of OTP.

(1)To ensure a clean development environment, directly re-run the launch.sh script file under the SDK directory, select the configuration file of 3917n, and then execute the command "make rp" in the generated workspace to rebuild the platform.

Note: In this example, 3917n is selected for testing. The workspace is generated after executing the launch.sh script and is by default located in out/rts3917n_base under the SDK directory

(2)After entering the workspace, execute the following commands:

```
make secure_boot M=2
```

Please refer to the specific meaning of the command *Compilation Commands* .

After the command is executed successfully, the following files will be generated in the rtskey folder under the SDK directory:

```
crypto_iv.bin                                #IV
crypto_key.bin                               #AES 256-bit key
verity_key0.key                              #The private key in PEM format␣
↪can generate the corresponding public key from the private key, which is␣
↪used for BootRom to verify U-Boot
verity_key0_pub.der.sha256.bin               #Hash of public key,need to write␣
↪OTP, dynamically generated at compile
verity_key1.crt                              #Self-signed public key␣
↪certificate, dynamically generated at compile
verity_key1.key                              #The private key in PEM format␣
↪can generate the corresponding public key from the private key, which is␣
↪used for U-boot to verify kernel
verity_key2.crt                              #Self-signed public key␣
↪certificate, dynamically generated at compile
verity_key2.key                              #The private key in PEM format␣
↪can generate the corresponding public key from the private key, which is␣
↪used for kernel to verify partitions with filesystem
```

(3)Burn the 256-bit AES key to Group1 of OTP and burn the IV to Group7 of OTP. Burning the SHA256 value of ROTPK to Group5 of OTP is actually burning the key and IV in the crypto boot verification step to OTP, and burning the SHA256 in the secure boot verification step to OTP.

①Read the contents of crypto_key.bin(aes-key), crypto_iv.bin(aes-iv), and verity_key0_pub.der.sha256.bin respectively on the PC using the following commands:

```
$cat rtskey/crypto_key.bin | xxd -ps -c 32
8f1324ed2a225c621e5bc757eac706b3d1b822981363a65e7d2a75b68e3ab766    /
↪/ aes-key
$ cat rtskey/crypto_iv.bin | xxd -ps
c20a7511058488a93d0d439ef97e09e7   // aes-iv
$ cat rtskey/verity_key0_pub.der.sha256.bin | xxd -p -c 32
232e21b095f79cf8e2eeb9626acb614c3b34728ad8ac5620787bc6607744d30a  //
↪hash
```

Note: During the actual usage process, please use the contents actually read as aes-key, aes-iv and Hash respectively

②On the device, use the tool otp_mfg to burn aes-key, aes-iv and hash into the corresponding Group:

```
$otp_mfg -w --ecc [aes-key] 16 // 16 represents the offset byte, and␣
↪--ecc indicates that the ECC value is calculated by the tool and␣
↪written into the Group
$otp_mfg -w --ecc [aes-iv] 208
$otp_mfg -r --pg // Check whether the write is correct. Here, the␣
↪check is to check whether aes-key and aes-iv are written to OTP␣
```

(continues on next page)

(continued from previous page)

```
↪correctly
$otp_mfg -w --bit 0 95 // Flag bit
$otp_mfg -w --bit 0 0 // Control bit
$otp_mfg -w --bit 0 6 // Control bit
$otp_mfg -w --ecc [hash] 144 // 144 represents the offset byte, and -
↪-ecc indicates that the ECC value is calculated by the tool and␣
↪written into the Group
$otp_mfg -r -pg // Check if the write is correct. Here, the check is␣
↪to verify whether the hash has been correctly written to the OTP
$otp_mfg -w --bit 0 97 // Flag bit
$otp_mfg -w --bit 0 4 // Control bit
```

Note: If it is the first time to write aes-key and aes-iv and the corresponding flag bit and control bit are not set, at this time, using the command otp_mfg -r –pg, the aes-key and aes-iv written to the OTP can be seen. If it is not the first write, the positions corresponding to AES-key are all 0. Because for Group1 where the AES key is located, after completing the write and setting the control position to 0, the otp_mfg tool will be unable to read the actual value of this Group and will be fixed to display all 0.

3.Update the image with the secure encrypted startup function to the device.

(1)It is necessary to check in the workspace whether the merge.ini file contains uboot, kernel, and rootfs. If the corresponding components are commented out, they need to be uncommented.

(2)Execute the command “make pack” in the workspace to package the image and generate the linux.bin file. The image file used for packaging is located in the “images” directory of the workspace, and the corresponding file is:

```
uboot image: fip.bin
kernel image: linux.crypted.itb
rootfs image: rootfs.squashfs.signed.crypted
```

(3)During the uboot command line stage of the device, use Ethernet and the command “update all” to download the linux.bin file to the device, For this step, please refer to the “Burning the Image via TFTP” section in the “U-boot” section of the “RTS3917 Series SDK Development User Guide”.

(4)Before restarting the device, it is necessary to cut off the power supply to the device first, and then set the working mode of the device to safe mode. For the safe mode of 3917n, GPIO7 to GPIO4 should be set to 0010 in sequence. After the Settings are completed, the device should be powered on and started again.

Note: For the working mode, you can refer to the “Working Mode” section of the “RTS3917_Series_Emergency_Mode_User_Guide_Manual”

Warning: If the working mode of the device is not set to safe mode, the device with the secure encrypted startup function enabled cannot start normally

(5)Enter the system and check whether the dm-crypt-newroot and dm-verity-newroot files are generated under the /dev/mapper directory. Moreover, the secure encrypted startup also has similar logs as follows:

①The log of ATF starting u-boot is as follows:

```
NOTICE:  Booting Trusted Firmware
NOTICE:  BL1: v2.3():V2.0_TO-3-g4ae38097
NOTICE:  BL1: Built : 13:39:47, Mar  7 2022
INFO:    BL1: RAM 0x19004000 - 0x1900f000
INFO:    Using crypto library 'mbed TLS'
INFO:    BL1: Loading BL2
INFO:    Using FIP in nor flash mode
INFO:    Loading image id=6 at address 0x80040000
INFO:    nor flash security mode
INFO:    Image id=6 loaded: 0x404551c - 0x404598b
INFO:    Using FIP in nor flash mode
INFO:    Loading image id=1 at address 0x404551c
INFO:    nor flash security mode
INFO:    Image id=1 loaded: 0x4000088 - 0x404551c
NOTICE:  BL1: Booting BL2
INFO:    Entry point address = 0x4000088
INFO:    SPSR = 0x1d3
```

②The log of u-boot starting the kernel is as follows:

```
## Loading kernel from FIT Image at 80400000 ...
   Using 'config@1' configuration
   Verifying Hash Integrity ... OK
   Trying 'kernel@1' kernel subimage
     Description:  Linux kernel
     Created:      2024-06-05  10:17:49 UTC
     Type:         Kernel Image
     Compression:  uncompressed
     Data Start:   0x804000c0
     Data Size:    3339928 Bytes = 3.2 MiB
     Architecture: ARM
     OS:           Linux
     Load Address: 0x81000000
     Entry Point:  0x81000000
     Sign algo:    sha256,rsa2048:verity_key1
     Sign value:  ␣
↪0d0c87dc171bc554139b5d0ea85d47a0f6b89fcd0a42ee8ca24f
                  ␣
↪f699d0af67a63a7546abe8eebf37e0feb33a4cfe4fceb572a256
                  ␣
↪d0f1aac21b28d8f65d5b55ad0f8ea91e6a86411c2ddb904c2362
                  ␣
↪98a2f89806d027a633acd1bc7d723bf28369439f8b8f9aac65b0
                  ␣
↪df4521ed94da80af497dae5dcbe7a670c14fc537f61626d4dfd4
                  ␣
↪841d79641e7bf490de6b8074e2e5a1388f465c2b4edabece65f6
                  ␣
↪32e1e4da689b9ad27db4335d76f8e7f0bff777c9c5b86cdc7793
                  ␣
↪45567b72488474fa4403e8484fa0d7e9ec7f222ba8325bb6b642
                  ␣
↪6c133cfe4a63ed524758304d1b80b2ccbc0ccedd7683c2bf80d7
                   bad513b122c654b878eaf4f62befc29f429584875b95
     Timestamp:    2024-06-05  10:17:49 UTC
   Verifying Hash Integrity ... sha256,rsa2048:verity_key1+ OK
## Flattened Device Tree blob at 8774ff34
```

(continues on next page)

(continued from previous page)

```
Booting using the fdt blob at 0x8774ff34
Loading Kernel Image
```

If all the above phenomena occur, it indicates that the verification of the secure encryption startup function has been successful.

Realtek Semiconductor Corp.
Headquarters
No. 2, Innovation Road II
Hsinchu Science Park, Hsinchu 300, Taiwan
Tel.: +886-3-578-0211. Fax: +886-3-577-6047
http://www.realtek.com